ALGORITMOS Y PROGRAMACIÓN II

Clase 3
Merge

Merge

Merge de listas en arreglos

Merge

La operación de merge consiste en generar una nueva estructura de datos (arreglos, listas) ordenada a partir de la mezcla de dos o más estructuras de datos previamente ordenadas.

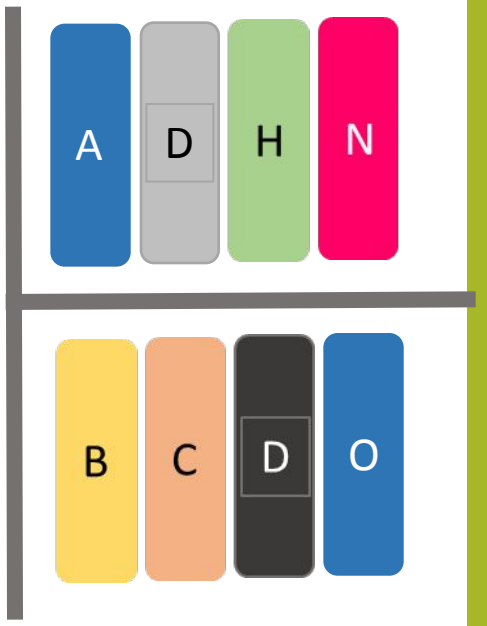
Las estructuras que se combinan guardan el mismo orden lógico interno (por ejemplo datos ordenados alfabéticamente)

Veremos un ejemplo de la vida real y como se asimila para poder resolverlo

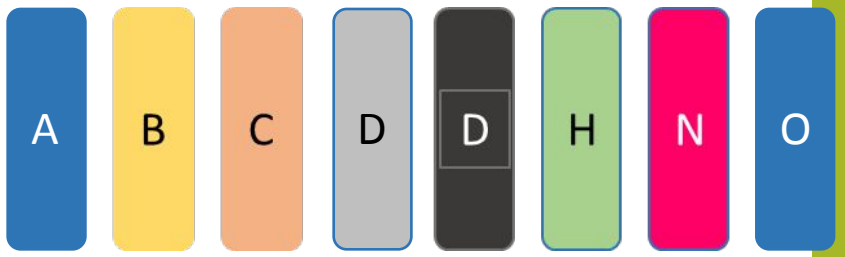
Merge

Juan y Paula viven juntos.

Juan y Paula tienen una biblioteca con dos estantes y en cada estante los libros se encuentran ordenados alfabéticamente.

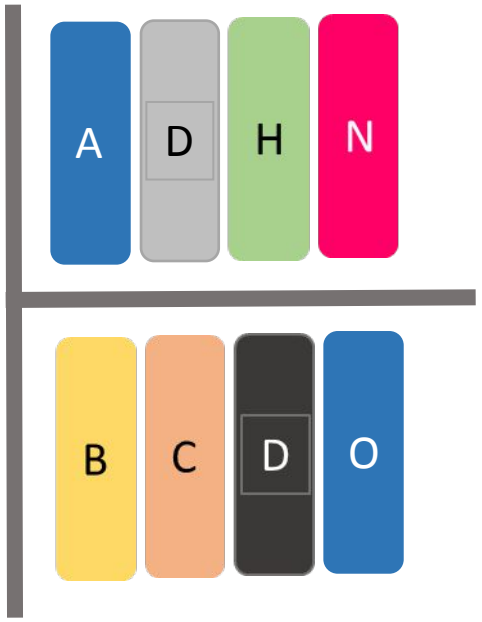


Quieren armar un único estante ordenado alfabéticamente.



Merge

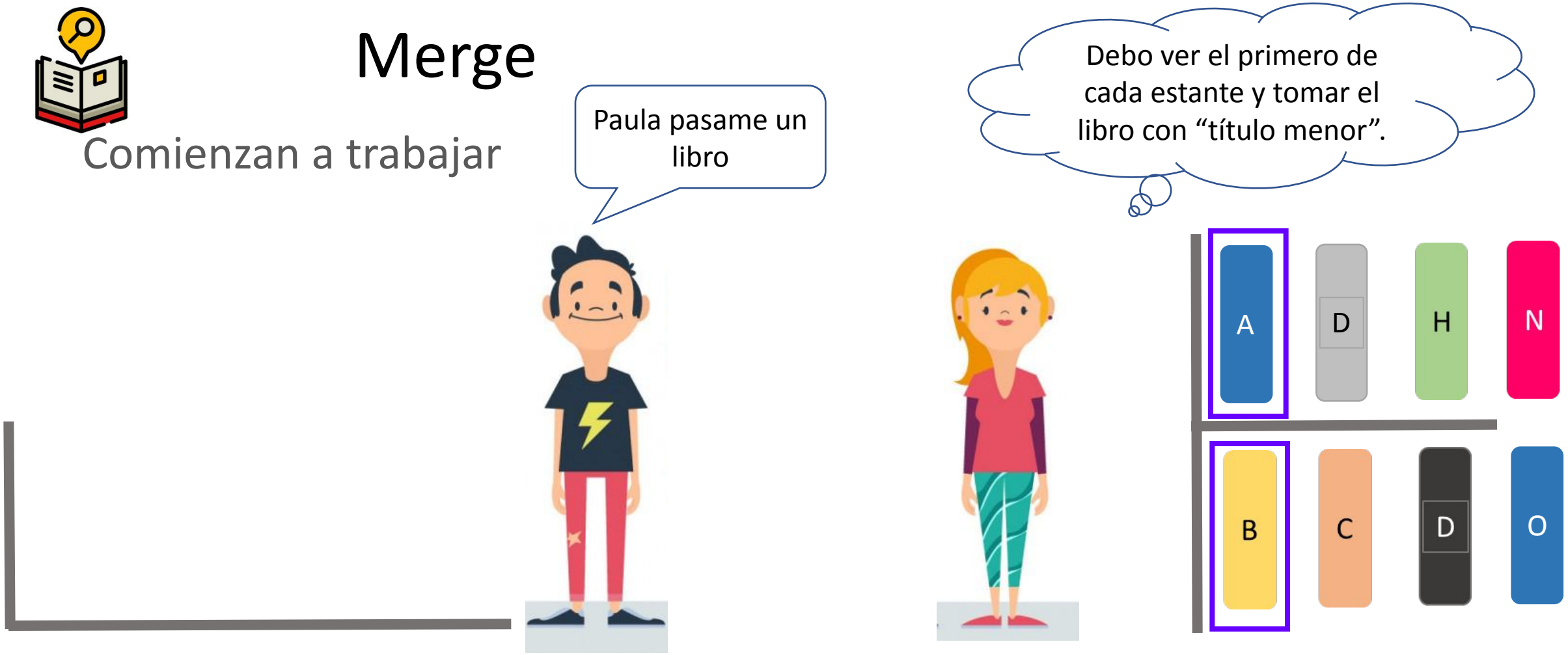
Juan se encargará de acomodar los libros en el nuevo estante.



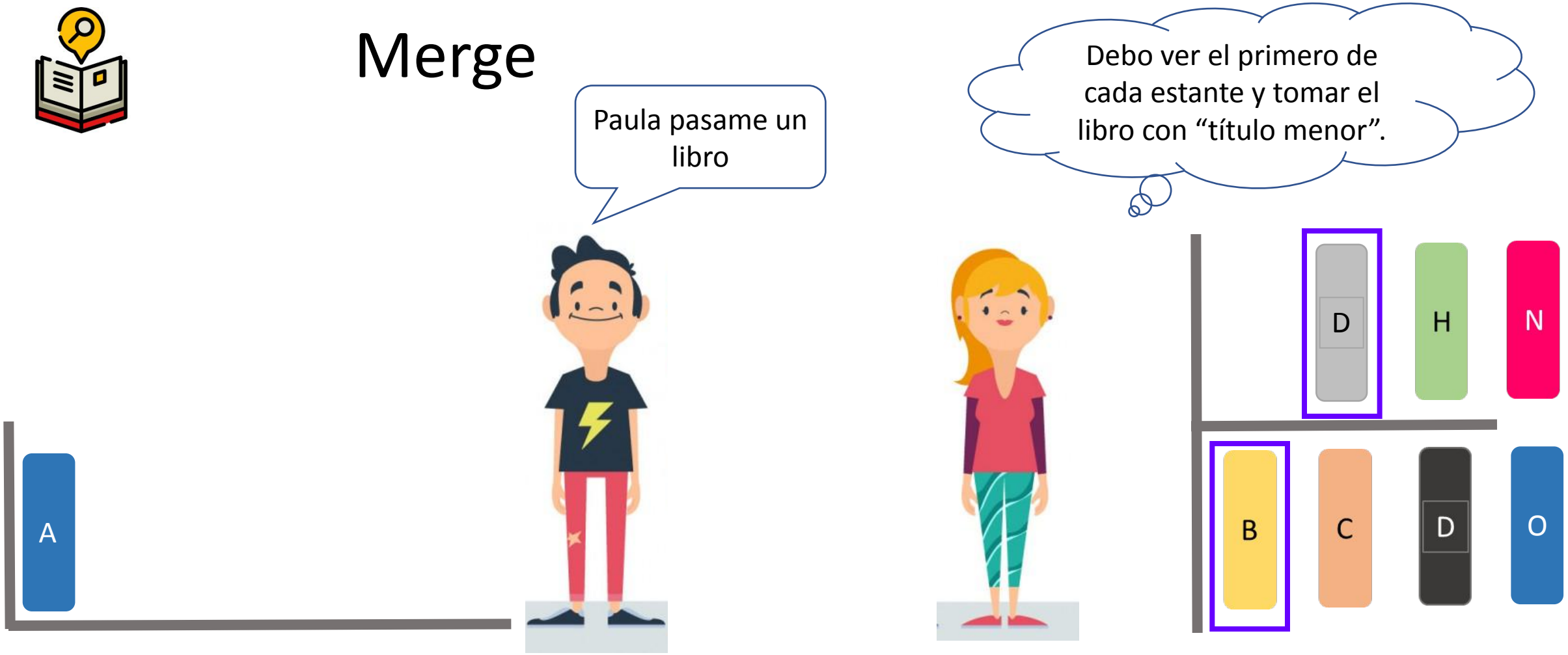
Paula le irá pasando los libros a Juan.

Merge

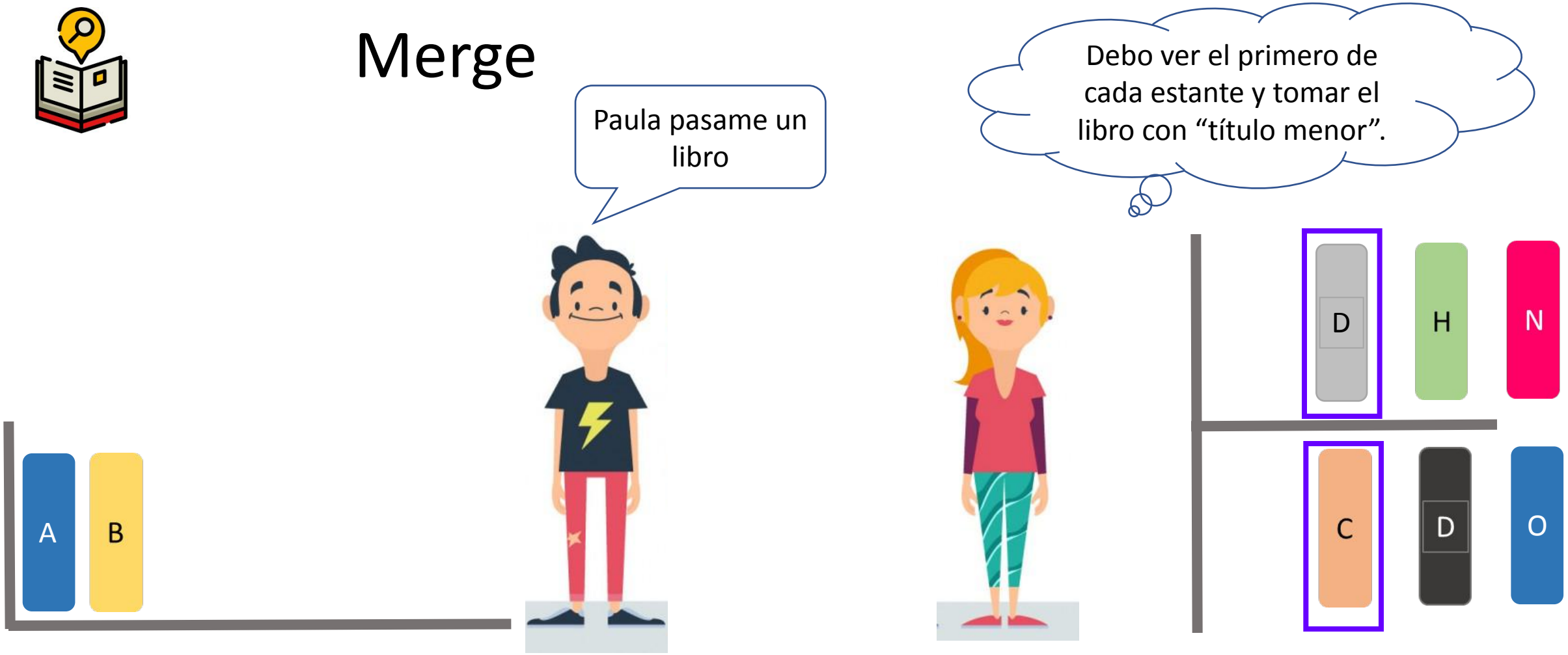
Comienzan a trabajar



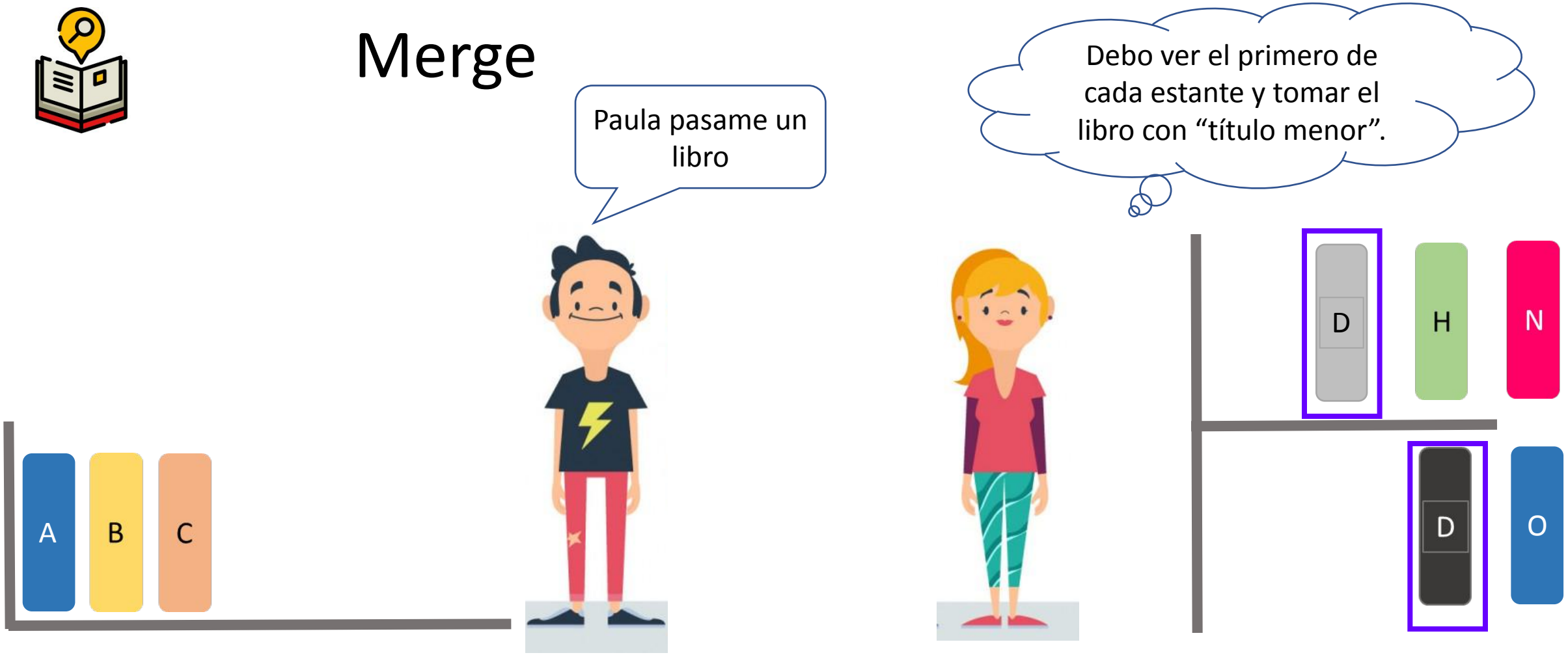
Merge


Paula pasame un libro
Debo ver el primero de cada estante y tomar el libro con “título menor”.
A
D
H
N
B
C
D
O

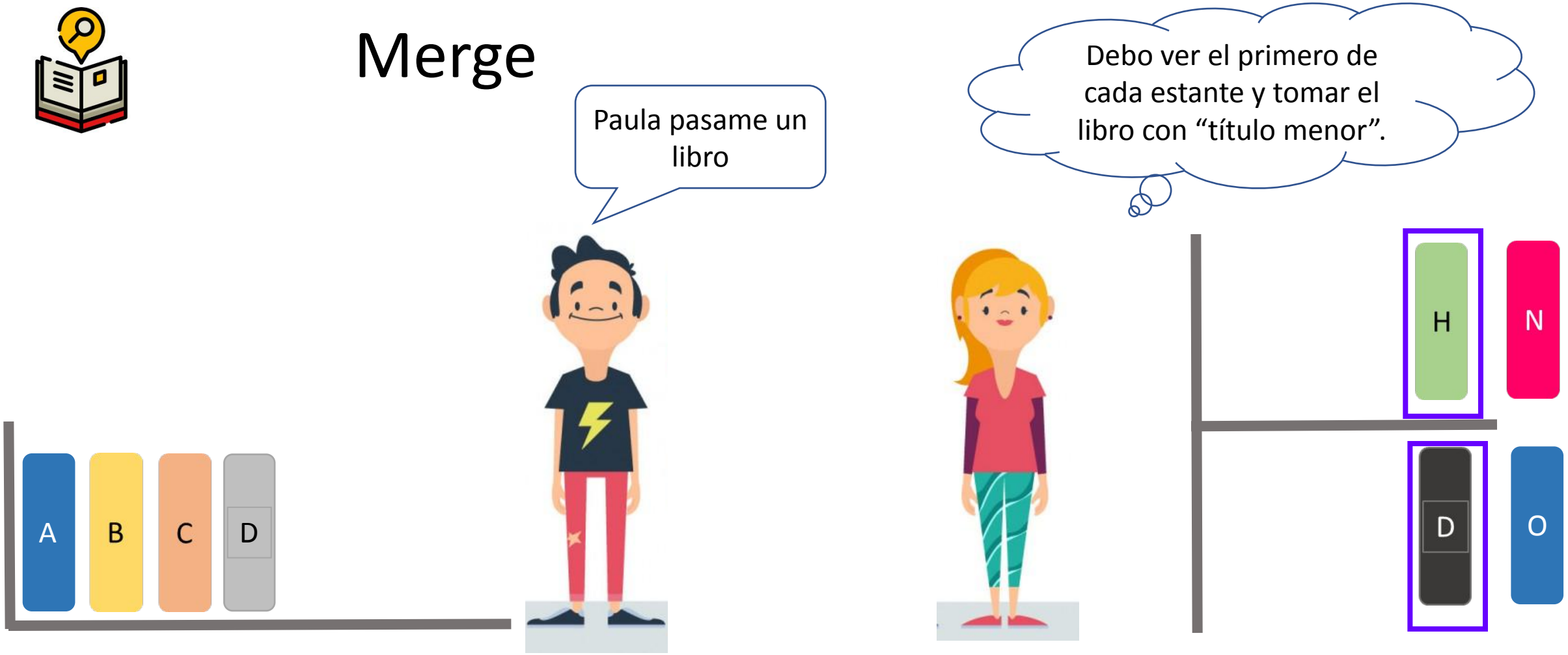
Merge


Paula pasame un libro
Debo ver el primero de cada estante y tomar el libro con “título menor”.
A
B
D
H
N
C
D
O

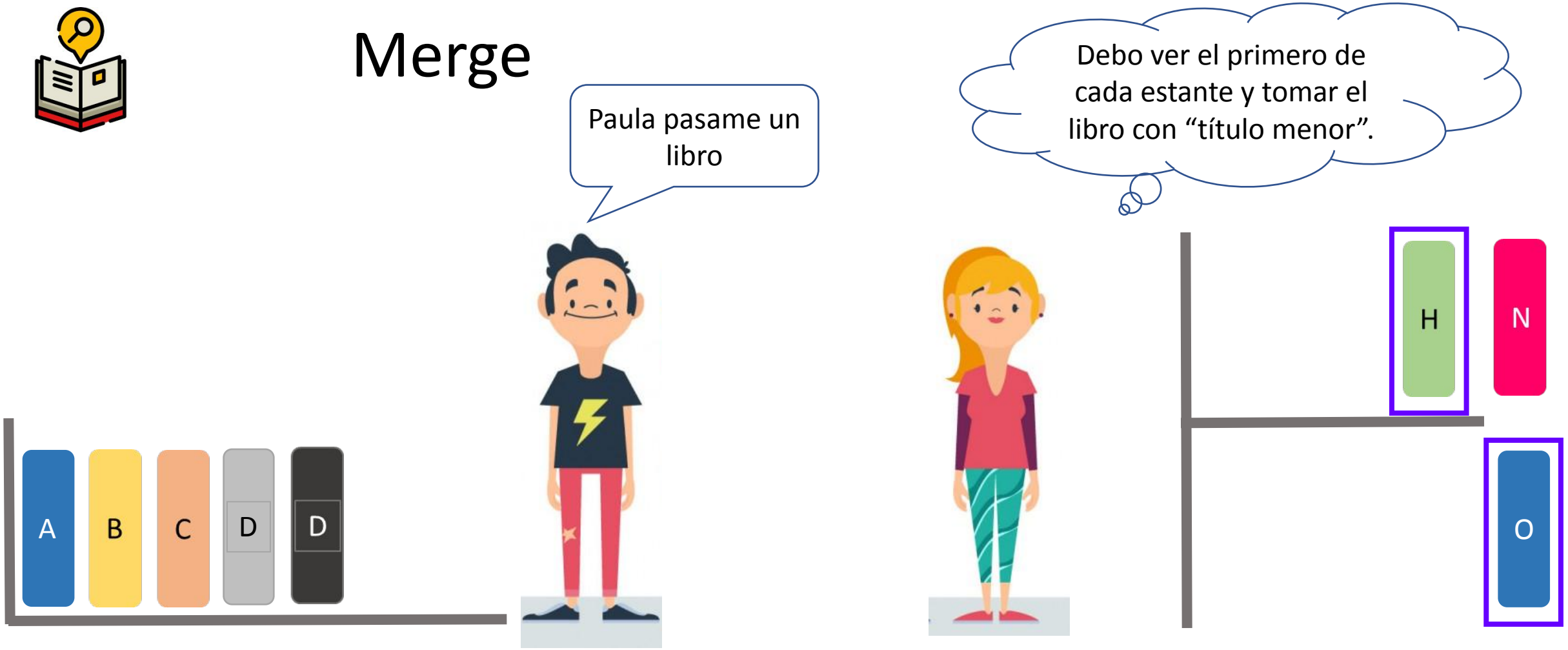
Merge


Paula pasame un libro
Debo ver el primero de cada estante y tomar el libro con “título menor”.
A
B
C
D
H
N
D
O

Merge


Paula pasame un libro
Debo ver el primero de cada estante y tomar el libro con “título menor”.
A
B
C
D
H
N
D
O

Merge

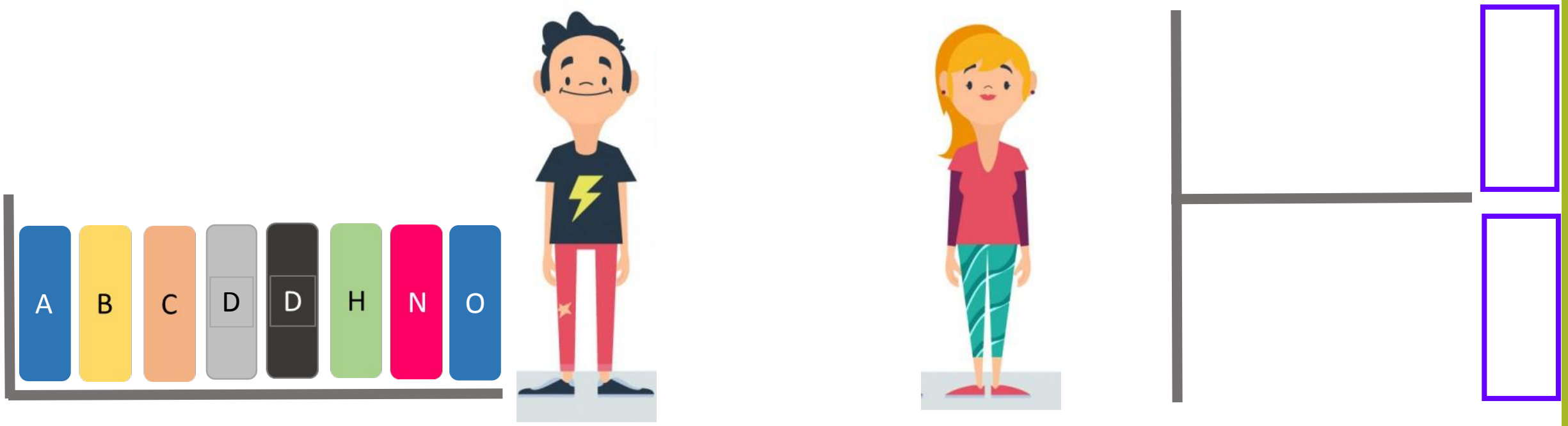

Paula pasame un libro
Debo ver el primero de cada estante y tomar el libro con “título menor”.
A
B
C
D
D
H
N
O

Merge

Siguiendo con la misma dinámica...

Merge

Se terminaron los libros de cada estante



Merge

Supongamos que se dispone de dos listas con libros (título, autor y año de edición) ordenados alfabéticamente por título y se pide generar una única lista de libros también ordenada alfabéticamente por título.

Libro Elemento de la Lista

Estante Lista

Biblioteca (2 estantes) dos Listas

```
Program merge;
Type libro = record
                titulo:string;
                autor: string;
                año: integer;
              end;
 listaLibros = specialize LinkedList<libro>;
Var estante1, estante2, estanteNuevo: listaLibros;

Begin
 cargarEstante (estante1);
 cargarEstante (estante2);
 merge (estante1,estante2,estanteNuevo);
End.
```

Merge

Supongamos que se dispone de dos listas con libros (título, autor y año de edición) ordenados alfabéticamente por título y se pide generar una única lista de libros también ordenada alfabéticamente por título.

```
Procedure merge (e1,e2: listaLibros; var eNuevo:listaLibros);

Begin
  mientras (los estantes tienen libros)
     Paula busca el mínimo
     Juan Guarda el mínimo
End.
```

Merge

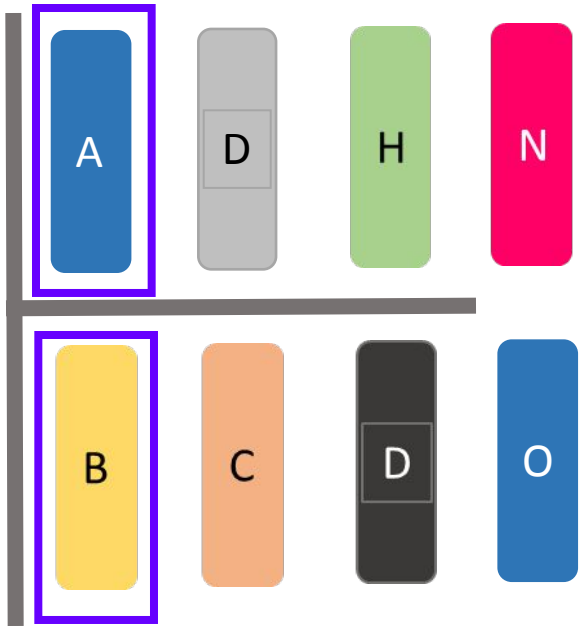
Supongamos que se dispone de dos listas con libros (título, autor y año de edición) ordenados alfabéticamente por título y se pide generar una única lista de libros también ordenada alfabéticamente por título.

Qué significa que Juan guarde el libro?

Qué significa que Paula busque el mínimo?

Como las listas de libros están ordenadas alfabéticamente por título, sólo debe mirar el primer elemento de cada lista, compararlos y determinar el menor entre estos.

Merge



PaulaBuscaMinimo...

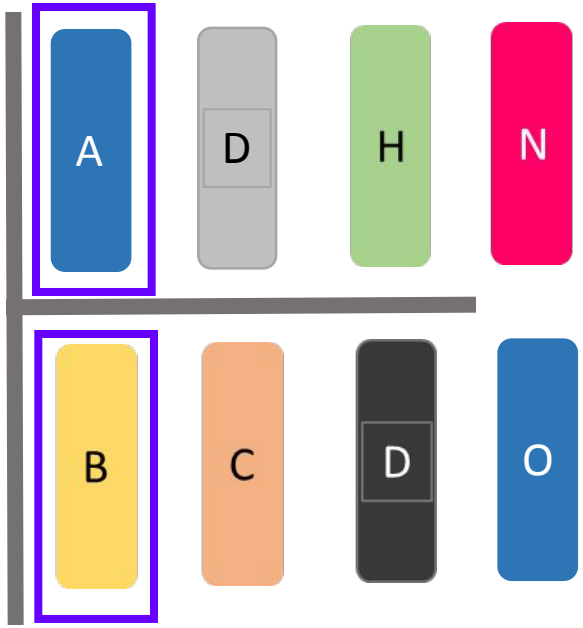
```
libroMinimo:=  valor_muy_alto
if (estante_1 noVacío) and (estante_2 noVacío)
then if (libroEnEstante_1 <= libroEnEstante_2 )
     then libroMinimo:= libroEnEstante_1
     else libroMinimo:= libroEnEstante_2

else if (estante_1 noVacío) {el estante 2 está vacío}
     then libroMinimo:= libroEnEstante_1
     else if (estante_2 noVacío) {el estante 1 está vacío}
          then libroMinimo:= libroEnEstante_2
```

Qué devuelve **libroMinimo** si los estantes están vacíos?

```
procedure minimo (var e1, e2:lista; var min:libro);
Begin
  min.titulo:= 'zzz';
  if (not e1.eol()) and (not e2.eol())
  then if (e1.current().titulo <= e2.current().titulo )
       then begin
              min:= e1.current();
              e1.next();
            end;
       else begin
              min:= e2.current();
              e2.next();
            end
  else if (not e1.eol())
       then begin
              min:= e1.current();
              e1.next();
            end
       else if (not e2.eol()) then
            begin
              min:= e2.current();
              e2.next();
            end;
end;
```

PaulaBuscaMinimo...



Merge

Supongamos que se dispone de dos listas con libros (título, autor y año de edición) ordenados alfabéticamente por título y se pide generar una única lista de libros también ordenada alfabéticamente por título.

```
Program Estantes2;
Type libro = record
               titulo:string;
               autor: string;
               año: integer;
             end;
 listaLibros = specialize LinkedList<libro>;
Var estante1, estante2, estanteNuevo: listaLibros;

Begin
 cargarEstante (estante1);
 cargarEstante (estante2);
 merge (estante1,estante2,estanteNuevo);
End.
```

```
procedure merge (e1, e2: listaLibros;
                 var eNuevo: listaLibros);

var min: libro;

Begin
  eNuevo:= listaLibros.create();
  e1.reset(); e2.reset();
  minimo (e1, e2, min);
  while (min.titulo <> ‘zzz’) do
    begin
     eNuevo.add (min);
     minimo (e1, e2, min);
    end;
End;
```

Actividades en Máquina

ACTIVIDAD 1.

Implementar:

A. Un módulo ***cargarEstante()*** que lea los libros aleatoriamente (de manera similar a como se generaban los inmuebles) y los almacene ordenados por título. La lectura finaliza cuando se genera un año de edición 2025.
B. Un módulo que reciba dos estructuras generadas con el módulo ***cargarEstante()*** y retorne una estructura de datos donde estén todos los libros almacenados y ordenados por título.
C. Un módulo que reciba la estructura generada en b y muestre los títulos de los libros almacenados.
D. Un programa que invoque a los módulos implementados y compruebe el correcto funcionamiento del mismo.

Merge

¿Qué pasa con los elementos de las dos listas originales?

¿Qué cambio hay que hacer para vaciar las listas originales a medida que se lleva a cabo el merge?

¿Qué pasa si tenemos 5 estantes en lugar de 2?

Merge

```
Program Estantes5;
const cantEstantes = 5;
Type libro = record
               titulo:string;
               autor: string;
               año: integer;
             end;

 listaLibros = specialize LinkedList<libro>;
 vector = array [1..cantEstantes] of listaLibros;

Var estantes: vector;
     estanteNuevo: listaLibros;
     i: integer;

Begin
 For i:= 1 to cantEstantes do
    cargarEstante (estantes [i]);
 merge (estantes,estanteNuevo);
End.
```

```
procedure merge (estantes: vector;
                 var eNuevo: listaLibros);

var min: libro;
    i: integer;
Begin
  eNuevo:= listaLibros.create();
  For i:= 1 to cantEstantes do
    estantes [i].reset();
  minimo (estantes, min);
  while (min.titulo <> ‘ZZZ’) do
    begin
     eNuevo.add (min);
     minimo (estantes, min);
    end;
End;
```

Merge

```
procedure minimo (var estantes: vector; var min:libro);
var i,posmin :integer;
Begin
  min.titulo:= ‘zzz’;

  For i:= 1 to cantEstantes do
     if not(estantes[i].eol()) then
        if estantes[i].current().titulo<= min.titulo then
         begin
            min:= estantes[i].current();
            posmin:=i
         end;

  if min.titulo<> ‘zzz’ then
     estantes[posmin].next();

end;
```

Actividades en Máquina

ACTIVIDAD 2

Un centro de distribución recibe paquetes de 6 lugares distintos. De cada paquete se conoce: código de paquete, código de lugar (entre 1 y 6), domicilio de envío y peso del paquete (Entre 1 Kg y 10 Kg en pasos de 500 gramos).
Implementar:

A. Un módulo llamado ***LeerPaquete()*** que lea los datos de paquetes aleatoriamente (de manera similar a como se generaban los inmuebles) y los almacene ordenados por peso y agrupados por código de lugar, en una estructura de datos adecuada. La lectura finaliza cuando se lee el código de paquete -1.
B. Un módulo que reciba la estructura generada en el punto a y retorne una estructura de datos donde estén todos los paquetes almacenados y ordenados por peso.
C. Un programa que invoque a los módulos implementados y compruebe el correcto funcionamiento del mismo.